

EvidenceDesk

AI Document Q&A RAG Knowledge Assistant

Full workflow, implementation, setup, evaluation and portfolio guide.

Prepared for Kavali Harshavardhan

1 October 2026

Project goal	Answer document questions using inspectable evidence.
Default experience	No-key lexical search returning cited excerpts.
Optional AI path	MiniLM semantic vectors plus Groq-generated answers.
Verified local results	16 tests passed; 23/25 answerable fixture questions retrieved the expected evidence.
Important distinction	92% evidence hit rate is not 92% AI answer accuracy.

Read this first

This guide explains the standalone EvidenceDesk repository, AI-Document-Q-A-RAG-Knowledge-Assistant. Run all project commands from its root folder, which contains app.py.

Live semantic inference, real LLM requests, Docker execution and public hosting have not been validated. API keys and a hosted deployment are not included.

Project destination

[Open the standalone project on GitHub](#)

Business problem and scope

What problem does it solve?

Users often search several documents manually to find a policy, product limit or procedure. The application indexes document passages, retrieves the passages that best match a question, and shows their origins. An optional language model turns those passages into a concise answer.

What RAG means

Retrieval-Augmented Generation combines a search stage with a generation stage. The language model receives the question and retrieved passages as context. The model is not retrained on the uploaded files. Updating the documents means rebuilding their index, not fine-tuning the model.

Mode	What happens
Lexical + excerpts	TF-IDF searches words and phrases. The application displays exact passage text with citations. This is retrieval, not generated AI.
Semantic + excerpts	A Sentence Transformer encodes passage meaning. Similarity search finds passages; the output remains exact excerpts.
Retrieval + Groq	Retrieved passages and the question are sent to an LLM. The response must include valid evidence numbers.

Included use case

Three fictional documents cover Cedar Labs people policies, security rules and an invented product called Atlas Notes. They contain no real employee or customer records. Use them for demonstrations and development tests.

Acceptance criteria

Users can upload supported files, build an index, ask standalone questions, inspect cited sources and clear the session. Empty or unsupported files produce errors. Low-scoring retrieval returns an insufficient-information message. Related-but-unanswerable questions remain a known weakness.

Complete project workflow

Stage	Action and purpose
01 Define requirements	Choose document types, question scope, privacy needs and success metrics.
02 Collect authorized documents	Start with the included fictional corpus or documents you have permission to process.
03 Extract and clean	Read PDF/DOCX/TXT; remove redundant spaces; retain source locations.
04 Split into chunks	Use 120-word windows with 25-word overlap, within extracted sections.
05 Build vector representations	Fit TF-IDF locally or encode passages with the optional MiniLM model.
06 Store the index	Use session memory for the app; JSON/NumPy persistence for the CLI.
07 Retrieve evidence	Represent a question with the same backend; rank cosine scores and apply a threshold.
08 Produce an answer	Show exact excerpts, or request a cited answer from Groq.
09 Validate and display	Check citation IDs; display filenames, locations, scores and full excerpts.
10 Evaluate and improve	Run automated tests and labeled questions; inspect failures and tune on a development split.
11 Publish and deploy	Commit code/docs to GitHub; hosting is a separate configuration and validation step.
12 Present the work	Use the demo, report and honest resume bullets; explain trade-offs and limitations.

Indexing: documents to searchable evidence

1. Extraction

PDF: pypdf reads each page and records its extraction page number. **DOCX:** python-docx reads paragraphs and tables. **TXT:** UTF-8 text is split at blank lines into sections. DOCX/TXT citations do not pretend to have page numbers.

The loader limits a file to 10 MB, a PDF to 250 pages, DOCX expanded content to 50 MB and extracted text to two million characters. The UI accepts up to 10 files. These are prototype limits, not a complete upload-security boundary.

2. Cleaning and chunking

Repeated spaces/tabs are normalized and empty text is discarded. Each extracted section is split into 120-word chunks with 25 words reused at the next boundary. For example, window one covers words 1-120 and window two starts at word 96. Overlap helps preserve facts around a boundary.

Short paragraphs stay as a single chunk. Sections are not merged. The sample corpus produces 22 chunks. The splitter counts words, not model tokens; unusually token-dense text can still be truncated by the semantic model.

3. Metadata and identifiers

Field	Purpose
id	Hash of filename, document content hash, location and chunk start; used to deduplicate exact chunks.
source	Document filename shown in the source viewer.
location	PDF page, DOCX paragraph/table, or TXT section.
text	The extracted chunk used for retrieval and evidence.

4. Persistence

The web app keeps an index in its current session. Rebuilding replaces it and clears the displayed chat history. The CLI writes index.json and, for semantic mode, vectors.npy. Lexical vectors are refitted on reload; semantic vectors are reloaded alongside the same query encoder. No pickle is loaded.

Retrieval: finding the relevant passages

Lexical baseline

TfidfVectorizer creates sparse vectors using unigrams and bigrams, English stopwords removal and sublinear term frequency. Rare, useful words receive more weight than common words. The fitted vectorizer must also transform the question so both vectors share the same vocabulary.

Optional semantic backend

Sentence Transformers loads sentence-transformers/all-MiniLM-L6-v2 and produces normalized vectors for both chunks and questions. Dense vectors can capture related meanings even when wording differs. The first use requires the optional dependency and model download; this path has not been evaluated live here.

Cosine similarity

Cosine similarity = dot(question vector, passage vector) / (question length x passage length). For normalized vectors this becomes a dot product. A larger value means a closer match in the representation, not a calibrated probability that the passage answers the question.

Setting	Implementation
Search engine	Exact vector comparison; sparse matrix multiplication or NumPy dense dot products.
Top-k	CLI/core default 4; UI default 3; evaluation uses 3.
Default threshold	0.12 lexical; 0.35 semantic. These are starting values, not calibrated production thresholds.
UI control	Users can change k and minimum similarity. Rebuild after changing retrieval backend.

Why retrieval sometimes fails

Synonyms, short queries, negation and broad policy terms can mislead a lexical method. Higher thresholds may reduce irrelevant passages while also losing valid answers. Evaluate on held-out questions before selecting a threshold.

Answer generation, citations and safeguards

Default: evidence excerpts

The application prefixes retrieved text with [1], [2] and so on. Each source expander shows the matching filename, location and similarity score. The answer is explicitly labeled as excerpts, so users know it is not a generated explanation.

Optional: Groq answer

The application sends a system instruction, the question and numbered evidence to the Groq chat-completions endpoint. The instruction requests answers supported only by the evidence, an insufficient-information response when needed, and JSON with answer and citations fields.

```
{"answer": "Employees receive 18 days [1]",  
  "citations": [1]}
```

The validator checks that the JSON has the expected types, citations are integer IDs within the retrieved range, and inline citation numbers match the citations list. Bad formats or IDs are rejected. Requests time out after 45 seconds; errors are shown without displaying the key.

What these checks do not prove

A valid citation number does not prove that a sentence is supported by that passage. The model may still infer unsupported facts. Prompt-injection instructions embedded in documents are treated as untrusted evidence by the prompt, but this alone is not a security guarantee.

Data handling

AI mode requires a server-side GROQ_API_KEY, an active GROQ_MODEL, and explicit UI consent to send the question and excerpts to Groq. API usage may incur charges. Do not put keys in GitHub. Do not expose private documents publicly without authentication and appropriate provider/data-handling review.

Illustrative answer

Question: How many annual leave days are available?

Evidence: The fictional people policy states 18 annual leave days per calendar year.

Output: A cited excerpt by default; a paraphrased cited answer only when AI mode is enabled.

Code walkthrough and repository map

File or folder	Responsibility
app.py	Streamlit upload, controls, index building, chat, source expanders, session reset and consent.
src/core.py	Chunk dataclass; extract(); chunk_document(); Index search/save/load; evidence_answer().
src/answering.py	Groq HTTP request, defensive prompt, result parsing and citation validation.
cli.py	Index a document folder and query a saved local index.
data/sample_documents/	Three fictional documents for reproducible demos.
evaluation/questions.json	25 answerable and 5 unsupported labeled questions.
evaluation/evaluate.py	Runs lexical retrieval and writes per-question results.
tests/test_pipeline.py	16 checks including loaders, overlap, persistence, retrieval, citation rejection and AppTest UI flow.
docs/	Project report, validation record, demo/interview notes, versions, deployment guide and this PDF.
requirements*.txt	Base, development and optional semantic dependencies.
Dockerfile	Base-dependency container recipe; not executed during validation.

A request through the code

The chat input sends a question to `Index.search()`. It returns (Chunk, score) pairs. `evidence_answer()` displays them directly, or `generate()` sends them to Groq. The UI stores the question, answer, timing and source labels in the session history. Previous messages are displayed but are not supplied to retrieval or the LLM; use complete standalone questions.

Run the project in VS Code on Windows

Install Python 3.11 or 3.12 and VS Code. Extract the project ZIP and open its main folder, which contains app.py. Use Terminal > New Terminal.

```
py -3.11 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m streamlit run app.py
```

If py -3.11 is unavailable, use python -m venv .venv with an installed compatible Python. Directly calling the virtual-environment interpreter avoids PowerShell activation-policy problems.

First successful demonstration

1. Open <http://localhost:8501>.
2. Keep the fictional sample documents enabled.
3. Click Build knowledge base.
4. Ask: How many annual leave days are available?
5. Expand [1] to inspect the policy section.
6. Ask: What is the orbital velocity of Neptune?
7. Upload your supported documents and rebuild to replace the sample index.

Enable optional semantic retrieval

```
.\.venv\Scripts\python.exe -m pip install -r requirements-semantic.txt
```

Select semantic and rebuild. Allow time and network access for the first model download. If model loading fails, select lexical and rebuild to keep using the local baseline.

Enable optional AI answers

Copy .env.example to .env. Enter your own GROQ_API_KEY and a currently available GROQ_MODEL from your provider console. Restart the app, enable Groq AI answers and select the consent checkbox. Never paste the key into source files or commit .env.

macOS / Linux

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python -m streamlit run app.py
```

Command line, tests and troubleshooting

Persistent local indexing

```
python cli.py index --documents data/sample_documents
python cli.py ask "How much does the Pro plan cost?"
# Optional, after semantic dependencies are installed:
python cli.py index --backend semantic
```

Commands assume you are using the project virtual environment. On Windows you may replace `python` with `.\venv\Scripts\python.exe`. The default index folder is `.local_index/`. It contains document text and must remain private. Rebuild the entire document folder after additions, edits or deletions.

Run verification

```
python -m pip install -r requirements-dev.txt
python -m pytest -q
python -m evaluation.evaluate
```

Problem	What to check
Module not found	Install requirements with the same interpreter used to launch the app. Run from the evidencedesk project root folder.
No readable text	The PDF may be scanned. Apply OCR outside this app, then upload an extracted-text version.
Unhelpful passages	Ask a specific standalone question, inspect sources, adjust <code>k/threshold</code> , or evaluate semantic retrieval.
Groq HTTP error	Check the key, active model ID, quota and network. Use excerpt mode while resolving the provider issue.
Invalid citation response	The model did not follow the required JSON/citation format. Review excerpts and try again.
New backend not active	Rebuild the knowledge base after selecting another retrieval method.

What is not automated

There is no OCR service, query-rewriting memory, reranker, login system or cloud data store. Docker and hosted deployment still require an actual build/run verification in the target environment.

Measured evaluation and limitations

Metric	Observed result
Automated tests	16 passed in the validated build.
Evidence hit rate at 3	23 of 25 answerable questions = 92%.
Unsupported-question abstention	3 of 5 questions = 60%.
Mean retrieval time	0.69 ms in the original small-corpus run; excludes startup, extraction and generation.
Live semantic/LLM evaluation	Not run. No generated answer accuracy or API-cost result is claimed.

The evaluator counts a hit when an expected source appears among the top three results and its text includes the annotated evidence phrase. The corpus and questions were authored together. This is a development fixture, not an independent benchmark or proof of real-world accuracy.

Two missed evidence questions

What is the minimum password length?
How do I cancel my subscription?

Two unsupported questions that retrieved related passages

Does the company pay overtime?
Are interns eligible for paid parental leave?

These results illustrate the difference between similarity and answerability. A related policy paragraph may be retrieved even when it does not answer the question. Excerpt mode displays that passage transparently; it does not establish a verified answer.

A stronger evaluation plan

Create an independent question set from authorized real documents. Split development and final test questions. Compare lexical, semantic and hybrid retrieval with identical top-k settings. Label answer correctness, citation support and appropriate refusals. Tune thresholds only on the development split. Measure full-request latency and provider usage after live generation is configured.

GitHub publication and deployment

Repository structure for this delivery

This project is published in its own repository. Application code is at the root, PDFs and diagrams are in docs/, and the Excel workbook is in evaluation/. The workflow `.github/workflows/tests.yml` runs tests from the repository root.

```
git clone https://github.com/harshakavali81-collab/AI-Document-Q-A-RAG-Knowledge-Assistant.git
cd AI-Document-Q-A-RAG-Knowledge-Assistant
python -m pip install -r requirements.txt
python -m streamlit run app.py
```

Local container deployment

```
docker build -t evidencedesk .
docker run --rm -p 8501:8501 evidencedesk
# Optional credentials from your local .env:
docker run --rm -p 8501:8501 --env-file .env evidencedesk
```

The supplied Docker recipe installs the lexical baseline dependencies only. Semantic mode requires installing `requirements-semantic.txt` in the image and providing enough memory/storage for the model. These Docker commands have not been executed in the delivery environment.

Hosted app

GitHub hosts the source code; it does not by itself run this Streamlit server. Use a host that can run a persistent Python process, install dependencies, route the app port and configure environment secrets. Add authentication before handling private documents. A public app URL is not included.

```
python -m streamlit run app.py \
  --server.address=0.0.0.0 --server.port=8501 \
  --server.headless=true
```

Before sharing a recruiter link

Verify the GitHub files and CI result, run the app locally, add your own screenshots or demo video, and explain which modes you tested. Do not describe the app as publicly deployed until an actual hosted URL works.

Interview, resume and learning checklist

60-second project explanation

I built EvidenceDesk, a document question-answering application in Python. It extracts text from PDF, DOCX and TXT files, creates chunks with source locations, and retrieves relevant evidence using vector similarity. The default mode returns cited excerpts; optional semantic retrieval and an LLM integration support a full RAG path. I tested extraction, persistence, citation checks and the chat workflow. On a small authored dataset, expected evidence appeared in the top three results for 23 of 25 answerable questions. Related-but-unanswerable questions revealed a limitation, so improving answerability evaluation is my next step.

Resume bullets - use after running and understanding it

- Built a Python document Q&A application with multi-format ingestion, source-aware chunking, vector retrieval and an interactive Streamlit evidence viewer.
- Implemented optional semantic retrieval and LLM generation, citation validation, and a 30-question evaluation supported by 16 automated tests.

Five-minute demonstration

Minute 1: Explain the problem and build the fictional corpus.

Minute 2: Ask the annual-leave question and inspect its source.

Minute 3: Ask a product question and an unsupported question.

Minute 4: Explain chunking, retrieval and optional generation.

Minute 5: Show tests and evaluation, including failures and next improvements.

Questions you should be able to answer

Why RAG instead of fine-tuning? Why chunk overlap? How does cosine similarity work? What is the difference between sparse and dense retrieval? What does a citation validator prove? Why can similarity fail on an unsupported question? What must change for private multi-user deployment?

Technical references

Streamlit: docs.streamlit.io/develop/api-reference

Sentence Transformers: sbert.net/docs/package_reference/sentence_transformer/model.html

Groq: console.groq.com/docs/api-reference

pypdf: pypdf.readthedocs.io/en/stable/user/extract-text.html

Implementation-specific facts and results in this guide come from the delivered source code and evaluation/results.json. Service/model availability may change; select an active model in your provider account.